INTERNSHIP REPORT

Study and familiarization with the nanosatellite flight software Exploration of the software architecture and prototyping of a Housekeeping module

Prepared by : RAHALI Youcef , LIOUANI Mostefa
Sector : Electronics and Industrial Computer Engineering

Supervisor : BOUZIANE Youcef

The Satellite Development Centre (CDS/ASAL)

March 2026

Contents

1 Introduction **2**

2 GlobusSat Project **2**

3 Definitions: Housekeeping (HK) and Whole Orbit Data (WOD) **2**
3.1 Housekeeping (HK) . . . 2
3.2 Whole Orbit Data (WOD) . . . 3

4 EM System Architecture **4**
4.1 Overview . . . 4
4.2 Software Layer Description . . . 4

5 EM Project File Tree **5**

6 Dependency Diagram — hk.c **5**
6.1 Dependency Summary Table . . . 6

7 Function Documentation **6**
7.1 GetTelemetryFilenameByType function . . . 6
7.2 TelemetryCollectorLogic function . . . 8
7.3 TelemetrySaveEPS function . . . 10
7.4 TelemetrySaveTRXVU function . . . 11
7.5 TelemetrySaveANT function . . . 13
7.6 TelemetrySaveSolarPanels function . . . 14
7.7 GetCurrentWODTelemetry function . . . 15
7.8 TelemetrySaveWOD function . . . 17
7.9 HousekeepingTask FreeRTOS Orchestration . . . 18

8 Implementation **19**
8.1 Simulation Environment . . . 19
8.2 FreeRTOS Task Configuration . . . 20
8.3 Task Periodicity and Orchestration . . . 21
8.4 Inter-task Communication via Queue . . . 22
8.5 Simulation Results . . . 22
8.6 WOD Telemetry Logging to CSV File . . . 24
8.6.1 Principle . . . 24
8.6.2 Script Description . . . 24

9 Conclusion **25**

©CDS/ ASAL

1 Introduction

This work is part of a national nanosatellite Earth observation programme, conducted within the Satellite Development Centre (CDS/ASAL). The satellite embarks an On-Board Computer (OBC) running a real-time operating system based on FreeRTOS. This report presents the analysis of the open-source **GlobusSat project**, which serves as a reference implementation for CubeSat housekeeping systems. The objective is to understand the architecture of the available source files and header files (.h), and to identify the functions required for the development of the Housekeeping (HK) module.

2 GlobusSat Project

The project is publicly available on GitHub and provides a complete reference implementation of a nanosatellite software architecture.
The satellite is built around the ISIS iOBC (On-Board Computer), which integrates an ARM9 processor running at up to 400 MHz. The iOBC runs FreeRTOS as its real-time operating system, providing task scheduling, mutex management, and inter-task communication primitives used extensively throughout the housekeeping module.
For power management, GlobusSat uses the ISIS iEPS (Electrical Power System), which provides two boards – the main board (MB) and the CDB board – both accessible via the I2C bus.
Communication is handled by the ISIS TRXVU transceiver, combining a UHF transmitter and a VHF receiver in a single unit.
The antenna system is the ISIS AntS, a deployable tape spring antenna with two independent sides (side A and side B) for redundancy. Both sides share the same I2C bus as the TRXVU and are monitored independently through the housekeeping module .

3 Definitions: Housekeeping (HK) and Whole Orbit Data (WOD)

3.1 Housekeeping (HK)

In satellite engineering, **Housekeeping** refers to the routine monitoring and reporting of a satellite's internal health and operational status. It is formally defined as **Service type 3** in the European space standard **ECSS-E-ST-70-41C** (Telemetry and Telecommand Packet Utilization), published by the European Cooperation for Space Standardization (ECSS) [8].

Typical housekeeping data collected in a nanosatellite includes:

Parameter	Description
Battery voltage	State of charge of the onboard battery
Bus currents	Current drawn on 3.3 V and 5 V power buses
Temperatures	Batteries, processors, solar panels
Power generation	Solar panel output current
Memory usage	Free and corrupt bytes on the filesystem
Reset counter	Number of OBC resets stored in FRAM
Error flags	Anomaly indicators from each subsystem

This data is periodically collected by the OBC and transmitted to the ground station via telemetry, enabling engineers to detect anomalies early and confirm satellite health [9].

3.2 Whole Orbit Data (WOD)

WOD stands for **Whole Orbit Data**. It is a compact housekeeping telemetry packet that is collected and stored onboard continuously throughout a full orbit — even when the satellite is not in contact with any ground station. When the satellite passes over a ground station, this accumulated data is downloaded in a single transfer.

WOD is particularly important for nanosatellites in Low Earth Orbit (LEO) because:

- Ground station contact windows are very short (typically 5–15 minutes per pass).
- Real-time housekeeping only covers a small fraction of the orbit.
- WOD provides a complete picture of the satellite's status over the full ∼90–100 minute orbital period.

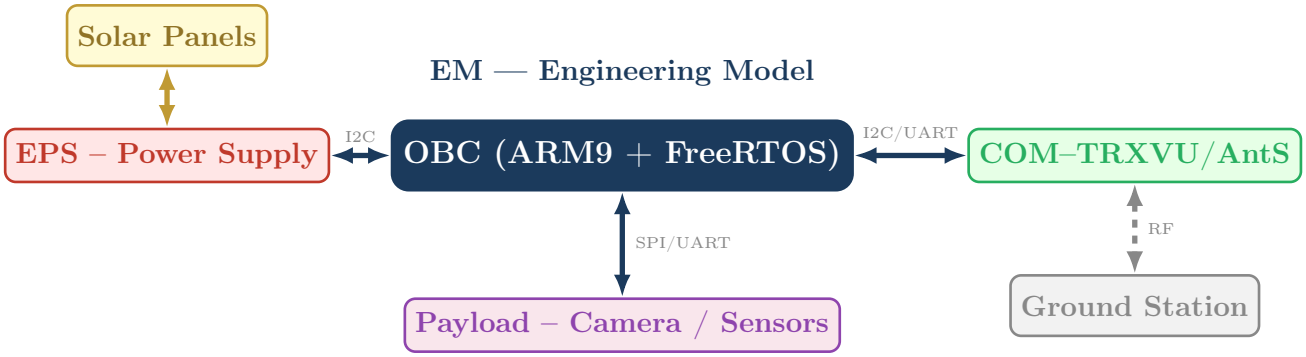
The use of WOD is confirmed in real CubeSat missions. In the **QB50** constellation (36 CubeSats deployed from the ISS in 2017), WOD was used to download magnetometer measurements collected over a full orbit and compare them against ground models [10].

Term	Definition
HK	Periodic real-time monitoring of satellite health parameters
WOD	Full-orbit summary of HK data, stored onboard and downloaded later

4 EM System Architecture

4.1 Overview

The EM (Engineering Model) is the engineering prototype of a nanosatellite. It integrates four main subsystems communicating through the iOBC, which acts as the central processing unit.



4.2 Software Layer Description

The embedded software of the EM is organised into four distinct layers:

Layer	Content	Role
Application Layer	`hk.c`, mission tasks	Mission logic
Subsystems Layer	`GomEPS.h`, `TRXVU.h`...	Drivers — subsystem interface
HAL Layer	`hal/errors.h`, I2C, SPI	iOBC hardware abstraction
FreeRTOS Kernel	Scheduler, tasks, queues	Real-time kernel, common to all layers

5 EM Project File Tree

The following tree presents the organisation of the header files provided in the Doxygen documentation:

```
OBC EM/
├── include/
│   └── satellite-subsystems/
│       ├── version/
│       │   └── version.h
│       ├── GomEPS.h ............................ EPS alimentation (GOMspace)
│       ├── IsisEPS.h ........................................ EPS alimentation
│       ├── IsisTRXVU.h ..................... COM VHF montant / UHF descendant
│       ├── IsisTRXUV.h ..................... COM UHF montant / VHF descendant
│       ├── IsisTxS.h ........................................ Émetteur S-band
│       ├── IsisAntS.h ...................................... Antenne déployable
│       ├── IsisMTQv1.h ................................... Magnetorquer version 1
│       ├── IsisMTQv2.h ................................... Magnetorquer version 2
│       ├── IsisSolarPanel.h ............................... Panneaux solaires v1
│       └── IsisSolarPanelv2.h ................... Panneaux solaires v2 (LTC ADC)
└── Application/
    └── hk.c ................................... Housekeeping Module (Globussat)
```

6 Dependency Diagram — hk.c

The `hk.c` module is at the centre of the application architecture. It includes (`#include`) the header files of each subsystem in order to call their telemetry collection functions:

Dependencies of hk.c

```
hk.c
  |-- #include GomEPS.h            ->  GomEpsGetHkData_general()
  |-- #include IsisEPS.h           ->  IsisEPS_getEngHKDataMB()
  |-- #include IsisTRXVU.h         ->  IsisTrxvu_rcGetTelemetryAll()
  |-- #include IsisTRXUV.h         ->  IsisTrxuv_imcGetTelemetryAll()
  |-- #include IsisSolarPanel.h    ->  IsisSolarPanel_getTemperature()
  |-- #include IsisSolarPanelv2.h ->  IsisSolarPanelv2_getTemperature
     ()
  |-- #include IsisMTQv1.h         ->  IsisMTQv1GetTemperature()
  |-- #include IsisAntS.h          ->  IsisAntS_getAlltelemetry()
  `-- #include IsisTxS.h           ->  IsisTxsGetHousekeepingData()
                |
                v
```

```
        HK_packet_t  -->  Ground Station
```

6.1 Dependency Summary Table

Header file	Subsystem	Main HK function	Data
GomEPS.h	EPS	GetHkData_general()	GOMspace power supply
IsisEPS.h	EPS	getEngHKDataMB()	ISIS power supply
IsisTRXVU.h	COM	rcGetTelemetryAll()	VHF/UHF transceiver
IsisTRXUV.h	COM	imcGetTelemetryAll()	UHF/VHF transceiver
IsisSolarPanel	Sensors	getTemperature()	Solar panels v1
IsisSolarPanel	Sensors	getTemperature() x5	Solar panels v2
IsisMTQv1.h	Sensors	GetTemperature()	Magnetorquer
IsisAntS.h	Antenna	getAlltelemetry()	Deployable antenna
IsisTxS.h	TX	GetHousekeepingData()	S-band transmitter

7 Function Documentation

7.1 GetTelemetryFilenameByType function

Description: Takes a telemetry type as input, and writes the corresponding filename string into the provided output buffer.

Input:

Parameter	Type	Description
tlm_type	tlm_type_t	The telemetry type to look up (e.g. tlm_eps_raw_mb, tlm_wod).

Output:

Parameter / Return	Description
`filename`	Buffer filled by the function with the filename string that corresponds to the given `tlm_type`.
`E_NO_SS_ERR`	Success — filename written into the buffer.
`E_INPUT_POINTER_NULL`	`filename` pointer is NULL, nothing written.
`E_PARAM_OUTOFBOUNDS`	`tlm_type` does not match any known type.

Source Code:

```c
int GetTelemetryFilenameByType(tlm_type_t tlm_type,
        char filename[MAX_F_FILE_NAME_SIZE])
{
    /* Check if output buffer is valid before writing into it */
    if (filename == NULL)
        return E_INPUT_POINTER_NULL;

    /* Match the telemetry type and write the filename into the buffer */
    switch(tlm_type)
    {
        case tlm_wod:           /* Whole Orbit Data */
            strcpy(filename, FILENAME_WOD_TLM);
            break;

        case tlm_eps_raw_mb:    /* EPS raw telemetry -- main board */
            strcpy(filename, FILENAME_EPS_RAW_MB_TLM);
            break;

        case tlm_eps_eng_mb:    /* EPS engineering telemetry -- main board */
            strcpy(filename, FILENAME_EPS_ENG_MB_TLM);
            break;

        case tlm_eps_raw_cdb:   /* EPS raw telemetry -- CDB board */
            strcpy(filename, FILENAME_EPS_RAW_CDB_TLM);
            break;

        case tlm_eps_eng_cdb:   /* EPS engineering telemetry -- CDB board */
            strcpy(filename, FILENAME_EPS_ENG_CDB_TLM);
            break;

        case tlm_solar:         /* Solar panel telemetry */
            strcpy(filename, FILENAME_SOLAR_PANELS_TLM);
```

```
            break;

        case tlm_tx:            /* Transmitter housekeeping */
            strcpy(filename, FILENAME_TX_TLM);
            break;

        case tlm_tx_revc:       /* Transmitter revision C data */
            strcpy(filename, FILENAME_TX_REVC);
            break;

        case tlm_rx:            /* Receiver housekeeping */
            strcpy(filename, FILENAME_RX_TLM);
            break;

        case tlm_rx_revc:       /* Receiver revision C data */
            strcpy(filename, FILENAME_RX_REVC);
            break;

        case tlm_rx_frame:      /* Last received frame from ground
            station */
            strcpy(filename, FILENAME_RX_FRAME);
            break;

        case tlm_antenna:       /* Antenna deployment telemetry */
            strcpy(filename, FILENAME_ANTENNA_TLM);
            break;

        default:                /* Unknown telemetry type */
            return E_PARAM_OUTOFBOUNDS;
    }

    return E_NO_SS_ERR; /* Success -- filename written */
}
```

7.2 TelemetryCollectorLogic function

Description: Called once at startup. Loads the telemetry save periods from FRAM, then creates one file per subsystem in the file system. If a file creation fails, the subsystem is marked inactive in `tlms_created` and will be skipped during telemetry collection.

Input: None.

Output: None. Results stored in `tlms_created` array — each entry set to `FALSE_8BIT` if its file was not created successfully.

Source Code:

```
void TelemetryCollectorLogic()
{
    FileSystemResult res; /* stores result of each c_fileCreate call */

    /* Load save periods for all subsystems from FRAM into RAM */
    FRAM_read((unsigned char*)tlm_save_periods,
              TLM_SAVE_PERIOD_START_ADDR,
              NUM_OF_SUBSYSTEMS_SAVE_FUNCTIONS * sizeof(time_unix));

    /* --- EPS files --- */
    res = c_fileCreate(FILENAME_EPS_RAW_MB_TLM, sizeof(ieps_rawhk_data_mb_t));
    SAVE_FLAG_IF_FILE_CREATED(tlm_eps_raw_mb)    /* mark inactive if failed */

    res = c_fileCreate(FILENAME_EPS_ENG_MB_TLM, sizeof(ieps_enghk_data_mb_t));
    SAVE_FLAG_IF_FILE_CREATED(tlm_eps_eng_mb)

    res = c_fileCreate(FILENAME_EPS_RAW_CDB_TLM, sizeof(ieps_rawhk_data_cdb_t));
    SAVE_FLAG_IF_FILE_CREATED(tlm_eps_raw_cdb)

    res = c_fileCreate(FILENAME_EPS_ENG_CDB_TLM, sizeof(ieps_enghk_data_cdb_t));
    SAVE_FLAG_IF_FILE_CREATED(tlm_eps_raw_cdb) /* BUG: should be tlm_eps_eng_cdb */

    /* --- TRXVU files --- */
    res = c_fileCreate(FILENAME_TX_TLM, sizeof(ISIStrxvuTxTelemetry));
    SAVE_FLAG_IF_FILE_CREATED(tlm_tx)

    res = c_fileCreate(FILENAME_TX_REVC, sizeof(ISIStrxvuTxTelemetry_revC));
    SAVE_FLAG_IF_FILE_CREATED(tlm_tx_revc)

    res = c_fileCreate(FILENAME_RX_TLM, sizeof(ISIStrxvuRxTelemetry));
    SAVE_FLAG_IF_FILE_CREATED(tlm_rx)

    res = c_fileCreate(FILENAME_RX_REVC, sizeof(ISIStrxvuRxTelemetry_revC));
    SAVE_FLAG_IF_FILE_CREATED(tlm_rx_revc)

    /* --- Antenna file --- */
```

```
    res = c_fileCreate(FILENAME_ANTENNA_TLM, sizeof(ISISantsTelemetry
        ));
    SAVE_FLAG_IF_FILE_CREATED(tlm_antenna)

    /* --- Solar panel file --- */
    res = c_fileCreate(FILENAME_SOLAR_PANELS_TLM,
                       sizeof(int32_t) * ISIS_SOLAR_PANEL_COUNT);
    SAVE_FLAG_IF_FILE_CREATED(tlm_solar)
}
```

7.3 TelemetrySaveEPS function

Description: Reads housekeeping telemetry from the EPS (Electrical Power System) via I2C and writes it to the file system. Collects four types of data: raw and engineering values from both the main board and the CDB board. A write to file only occurs if the I2C read succeeded.

Input: None.

Output: None. Data written directly to FRAM files.

File written	Content
FILENAME_EPS_RAW_MB_TLM	Raw housekeeping — main board.
FILENAME_EPS_ENG_MB_TLM	Engineering housekeeping — main board.
FILENAME_EPS_RAW_CDB_TLM	Raw housekeeping — CDB board.
FILENAME_EPS_ENG_CDB_TLM	Engineering housekeeping — CDB board.

Source Code:

```
void TelemetrySaveEPS()
{
    int err = 0;                          /* result of each I2C read
        call */
    ieps_statcmd_t cmd;                   /* EPS response status (
        command code,
                                              error bits, new data flag)
                                                  */
    ieps_board_t brd = ieps_board_cdb1; /* select CDB board 1 for CDB
         calls */

    /* --- Main board raw telemetry --- */
    ieps_rawhk_data_mb_t tlm_mb_raw;    /* buffer for raw HK data (ADC
         counts) */
    err = IsisEPS_getRawHKDataMB(EPS_I2C_BUS_INDEX, &tlm_mb_raw, &cmd
```

```
        );
    if (err == 0)         /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_EPS_RAW_MB_TLM, &tlm_mb_raw);

    /* --- Main board engineering telemetry --- */
    ieps_enghk_data_mb_t tlm_mb_eng;   /* buffer for engineering HK data
                                          (converted: volts, amperes, celsius) */
    err = IsisEPS_getEngHKDataMB(EPS_I2C_BUS_INDEX, &tlm_mb_eng, &cmd);
    if (err == 0)                      /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_EPS_ENG_MB_TLM, &tlm_mb_eng);

    /* --- CDB board raw telemetry --- */
    ieps_rawhk_data_cdb_t tlm_cdb_raw; /* buffer for CDB raw HK data */
    err = IsisEPS_getRawHKDataCDB(EPS_I2C_BUS_INDEX, brd, &tlm_cdb_raw, &cmd);
    if (err == 0)                      /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_EPS_RAW_CDB_TLM, &tlm_cdb_raw);

    /* --- CDB board engineering telemetry --- */
    ieps_enghk_data_cdb_t tlm_cdb_eng; /* buffer for CDB engineering HK data */
    err = IsisEPS_getEngHKDataCDB(EPS_I2C_BUS_INDEX, brd, &tlm_cdb_eng, &cmd);
    if (err == 0)                      /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_EPS_ENG_CDB_TLM, &tlm_cdb_eng);
}
```

7.4 TelemetrySaveTRXVU function

Description: Reads housekeeping telemetry from the TRXVU (Transceiver) via I2C and writes it to the file system. Collects four types of data: transmitter and receiver telemetry for both standard and revision C hardware. A write to file only occurs if the I2C read succeeded.

Input: None.

Output: None. Data written directly to FRAM files.

File written	Content
FILENAME_TX_TLM	Transmitter housekeeping — standard revision.
FILENAME_TX_REVC	Transmitter housekeeping — revision C hardware.
FILENAME_RX_TLM	Receiver housekeeping — standard revision.
FILENAME_RX_REVC	Receiver housekeeping — revision C hardware.

Source Code:

```
void TelemetrySaveTRXVU()
{
    int err = 0; /* result of each I2C read call */

    /* --- Transmitter standard revision --- */
    ISIStrxvuTxTelemetry tx_tlm;  /* buffer for TX housekeeping data */
    err = IsisTrxvu_tcGetTelemetryAll(ISIS_TRXVU_I2C_BUS_INDEX, &tx_tlm);
    if (err == 0)                 /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_TX_TLM, &tx_tlm);

    /* --- Transmitter revision C --- */
    ISIStrxvuTxTelemetry_revC revc_tx_tlm; /* buffer for TX revC data */
    err = IsisTrxvu_tcGetTelemetryAll_revC(ISIS_TRXVU_I2C_BUS_INDEX,
            &revc_tx_tlm);
    if (err == 0)                 /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_TX_REVC, &revc_tx_tlm);

    /* --- Receiver standard revision --- */
    ISIStrxvuRxTelemetry rx_tlm;  /* buffer for RX housekeeping data */
    err = IsisTrxvu_rcGetTelemetryAll(ISIS_TRXVU_I2C_BUS_INDEX, &rx_tlm);
    if (err == 0)                 /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_RX_TLM, &rx_tlm);

    /* --- Receiver revision C --- */
    ISIStrxvuRxTelemetry_revC revc_rx_tlm; /* buffer for RX revC data */
```

```
    err = IsisTrxvu_rcGetTelemetryAll_revC(ISIS_TRXVU_I2C_BUS_INDEX,
            &revc_rx_tlm);
    if (err == 0)                    /* write only if I2C read succeeded */
        c_fileWrite(FILENAME_RX_REVC, &revc_rx_tlm);
}
```

7.5 TelemetrySaveANT function

Description: Reads housekeeping telemetry from both sides of the antenna system (side A and side B) via I2C and writes both records to the file system. Both reads must succeed — if either one fails, nothing is written.

Input: None.

Output: None. Data written directly to FRAM file.

File written	Content
FILENAME_ANTENNA_TLM	Side A telemetry record (first write).
FILENAME_ANTENNA_TLM	Side B telemetry record (second write).

Source Code:

```
void TelemetrySaveANT()
{
    int err = 0; /* cumulative result of both I2C read calls */

    /* Two buffers: one for each antenna side */
    ISISantsTelemetry ant_tlmA, ant_tlmB;

    /* Read all telemetry from antenna side A via I2C */
    err = IsisAntS_getAlltelemetry(ISIS_TRXVU_I2C_BUS_INDEX,
            isisants_sideA, &ant_tlmA);

    /* Read all telemetry from antenna side B, add result to err */
    /* if either read fails, err != 0 and nothing will be written */
    err += IsisAntS_getAlltelemetry(ISIS_TRXVU_I2C_BUS_INDEX,
            isisants_sideB, &ant_tlmB);

    /* Write only if both reads succeeded (err == 0) */
    if (err == 0)
    {
        c_fileWrite(FILENAME_ANTENNA_TLM, &ant_tlmA); /* save side A record */
        c_fileWrite(FILENAME_ANTENNA_TLM, &ant_tlmB); /* save side B
```

```
            record */
    }
}
```

7.6 TelemetrySaveSolarPanels function

Description: Reads the temperature of all 9 solar panels one by one. First checks the solar panel system is awake, then reads each panel. Each read returns `ISIS_SOLAR_PANEL_STATE_AWAKE` = 2 on success. Writes to file only if all 9 panels returned `AWAKE`, i.e. `err` $= 2 \times 9 = 18$

Input: None.

Output: None. Data written directly to FRAM file.

File written	Content
`FILENAME_SOLAR_PANELS_TLM`	Array `t[0..8]` of 9 temperatures (`int32_t`), one per panel. Range: -273.16 to $8192\,°C$, resolution 1/1024.

Source Code:

```
void TelemetrySaveSolarPanels()
{
    int err = 0;            /* cumulative result of all read calls */
    int32_t t[ISIS_SOLAR_PANEL_COUNT];  /* Temperature buffer, one
        int32_t per panel (range -273.16 to 8192 C) */

    uint8_t fault;  /* Fault status byte returned by each individual
        panel read */

    /* Check global solar panel system state before reading */
    /* NOINIT=0, SLEEP=1, AWAKE=2 -- only proceed if AWAKE */
    if (IsisSolarPanelv2_getState() == ISIS_SOLAR_PANEL_STATE_AWAKE)
    {
        /* Read temperature from each panel, returns AWAKE=2 on
            success */
        err =  IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_0,&t
            [0],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_1,&t
            [1],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_2,&t
            [2],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_3,&t
            [3],&fault);
```

```
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_4,&t[4],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_5,&t[5],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_6,&t[6],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_7,&t[7],&fault);
        err += IsisSolarPanelv2_getTemperature(ISIS_SOLAR_PANEL_8,&t[8],&fault);

        /* All 9 panels awake: err = AWAKE(2) * 9 panels = 18 */
        /* If any panel returned SLEEP(1) or NOINIT(0), err < 18, skip */
        if (err == ISIS_SOLAR_PANEL_STATE_AWAKE * ISIS_SOLAR_PANEL_COUNT)
        {
            /* Write entire temperature array as one record */
            c_fileWrite(FILENAME_SOLAR_PANELS_TLM, t);
        }
    }
    /* If system is not awake, skip everything -- do not write stale data */
}
```

7.7 GetCurrentWODTelemetry function

Description: Fills a `WOD_Telemetry_t` structure with a current snapshot of the satellite state. WOD (Whole Orbit Data) aggregates memory status, mission time, EPS power values, and reset counter into one packet for transmission to the ground station.

Input:

Parameter	Type	Description
`wod`	`WOD_Telemetry_t*`	Pointer to the struct to be filled by the function.

Output: None. The `wod` struct is filled directly with the following fields:

Field written	Content
wod->free_memory	Free space remaining in the file system.
wod->corrupt_bytes	Number of bad/corrupt bytes in the file system.
wod->sat_time	Current satellite mission time (Unix timestamp).
wod->vbat	Battery voltage from EPS CDB board.
wod->current_3V3	Current on the 3.3V power bus.
wod->current_5V	Current on the 5V power bus.
wod->volt_3V3	Voltage on the 3.3V power bus.
wod->volt_5V	Voltage on the 5V power bus.
wod->charging_power	Power generated by solar panels.
wod->consumed_power	Power consumed by the satellite.
wod->number_of_resets	Total reset counter read from FRAM.

Source Code:

```c
void GetCurrentWODTelemetry(WOD_Telemetry_t *wod)
{
    /* Reject NULL pointer -- nothing to fill */
    if (NULL == wod)
        return;

    /* Zero the entire struct -- no garbage values if a read fails */
    memset(wod, 0, sizeof(*wod));
    int err = 0;

    /* --- Memory section --- */
    FN_SPACE space = { 0 };           /* FAT file system space descriptor */
    int drivenum = f_getdrive();      /* get current active drive number */

    err = f_getfreespace(drivenum, &space); /* read free and bad space */
    if (err == F_NO_ERROR) {
        wod->free_memory   = space.free; /* free bytes in file system */
        wod->corrupt_bytes = space.bad;  /* corrupt/bad bytes detected */
    }

    /* --- Time section --- */
    time_unix current_time = 0;
    Time_getUnixEpoch(&current_time);     /* read current mission time */
    wod->sat_time = current_time;         /* store Unix timestamp in
```

```
        WOD */

    /* --- EPS power section --- */
    ieps_statcmd_t cmd;                    /* EPS response status */
    ieps_enghk_data_mb_t  hk_tlm;         /* EPS engineering HK -- main board */
    ieps_enghk_data_cdb_t hk_tlm_cdb;     /* EPS engineering HK -- CDB board */
    ieps_board_t board = ieps_board_cdb1;/* select CDB board 1 */

    /* Read engineering HK from both EPS boards via I2C */
    err =  IsisEPS_getEngHKDataCDB(EPS_I2C_BUS_INDEX,board,&hk_tlm_cdb,&cmd);
    err += IsisEPS_getRAEngHKDataMB(EPS_I2C_BUS_INDEX, &hk_tlm, &cmd);

    if (err == 0) {  /* write power fields only if both reads succeeded */
        wod->vbat            = hk_tlm_cdb.fields.bat_voltage;  /* battery voltage */
        wod->current_3V3     = hk_tlm.fields.obus3V3_curr;     /* 3.3V bus current */
        wod->current_5V      = hk_tlm.fields.obus5V_curr;      /* 5V bus current */
        wod->volt_3V3        = hk_tlm.fields.obus3V3_volt;     /* 3.3V bus voltage */
        wod->volt_5V         = hk_tlm.fields.obus5V_volt;      /* 5V bus voltage */
        wod->charging_power = hk_tlm.fields.pwr_generating;   /* solar power in */
        wod->consumed_power = hk_tlm.fields.pwr_delivering;   /* power consumed */
    }

    /* --- Reset counter section --- */
    /* Read number of resets directly from FRAM (survives power cycles) */
    FRAM_read((unsigned char*)&wod->number_of_resets,
              NUMBER_OF_RESETS_ADDR, NUMBER_OF_RESETS_SIZE);
}
```

7.8 TelemetrySaveWOD function

Description: Creates a `WOD_Telemetry_t` struct, fills it with the current satellite state by calling `GetCurrentWODTelemetry`, then writes it as one record to the WOD file in FRAM.

Input: None.

Output: None. Data written directly to FRAM file.

File written	Content
FILENAME_WOD_TLM	One WOD_Telemetry_t record containing battery voltage, bus currents, bus voltages, power, mission time, free memory, corrupt bytes, and reset counter.

Source Code:

```
void TelemetrySaveWOD()
{
    /* Create WOD struct in RAM, initialize all fields to zero */
    WOD_Telemetry_t wod = { 0 };

    /* Fill the struct with current satellite state:
       battery voltage, currents, voltages, power,
       mission time, free memory, corrupt bytes, reset counter */
    GetCurrentWODTelemetry(&wod);

    /* Write the filled struct as one record to the WOD file in FRAM
       */
    c_fileWrite(FILENAME_WOD_TLM, &wod);
}
```

7.9 HousekeepingTask FreeRTOS Orchestration

Description

`HousekeepingTask` is the central FreeRTOS task of the Housekeeping module. It is created once at startup by the main scheduler and runs indefinitely. At each wake-up, it calls `TelemetryCollectorLogic()`, which compares the current Unix timestamp (`time_unix`) against the last save timestamp of each subsystem, and invokes the corresponding `TelemetrySave*` function only if the configured period has elapsed.

The save periods are loaded from FRAM at startup by `createfiletelemtry()`, making them persistent across OBC resets.

Input: None (`void *pvParameters` is ignored).

Output: None. Data is written directly to FRAM files by each `TelemetrySave*` function via `c_fileWrite`.

Note on grouped saves. `TelemetrySaveEPS()` performs four I2C reads in a single call, collecting raw and engineering telemetry from both the main board (MB) and the CDB board. The two indices `tlm_eps_raw_mb` and `tlm_eps_eng_mb` therefore trigger two separate timed calls to the same function. The same applies to `TelemetrySaveTRXVU()`, which is called independently for `tlm_tx` and `tlm_rx`.

Note on WOD save. `TelemetrySaveWOD()` does not only write a file it internally calls `GetCurrentWODTelemetry()`, which performs active I2C reads to both EPS boards to collect power parameters at save time. The WOD packet therefore reflects the satellite state at the *moment of collection*, not a cached value.

8 Implementation

This section describes the prototyping and simulation of the Housekeeping module developed during this internship. Since the ISIS iOBC hardware was not available for direct testing,we used arduino-based simulation to validate the logic of the telemetry collection and file writing processes. The following subsections detail the simulation environment, task configuration, and the implementation of key functions.

8.1 Simulation Environment

The simulation was implemented on an **Arduino Mega 2560** (ATmega2560, 8-bit AVR, 16 MHz) using the *FreeRTOS by Richard Barry* library, available in the Arduino IDE Library Manager. Although the Arduino Mega is architecturally different from the ISIS iOBC (ARM9, 32-bit), this platform was chosen for its accessibility and for the availability of a FreeRTOS port, allowing the task scheduling logic to be validated within the one-week internship constraint.
All hardware-dependent calls ISIS EPS drivers, TRXVU, antenna, solar panel sensors, FRAM read/write, and the FAT file system were replaced by **stub functions** returning randomised values within realistic ranges. The `c_fileWrite()` function was replaced by a `Serial.print()` call to display results on the Arduino IDE Serial Monitor.

Table 2: Hardware calls replaced by stubs in the simulation

Real function (iOBC)	Stub replacement	Simulated value
`IsisEPS_getEngHKDataCDB()`	`random()`	Vbat: 3600–4200 mV
`IsisEPS_getRAEngHKDataMB()`	`random()`	Volt_3V3: 3250–3350 mV
`IsisTrxvu_tcGetTelemetryAll()`	`random()`	TX power: −30 to 0 dBm
`IsisSolarPanelgetTemperature()`	`random()`	Temperature: −20 to 80 °C
`Time_getUnixEpoch()`	`millis()/1000`	Seconds since boot
`FRAM_read()`	Fixed value	Reset counter: 3
`f_getfreespace()`	`random()`	Free memory: 100 000–512 000 B
`c_fileWrite()`	`Serial.print()`	Filename printed toSerialMonitor

8.2 FreeRTOS Task Configuration

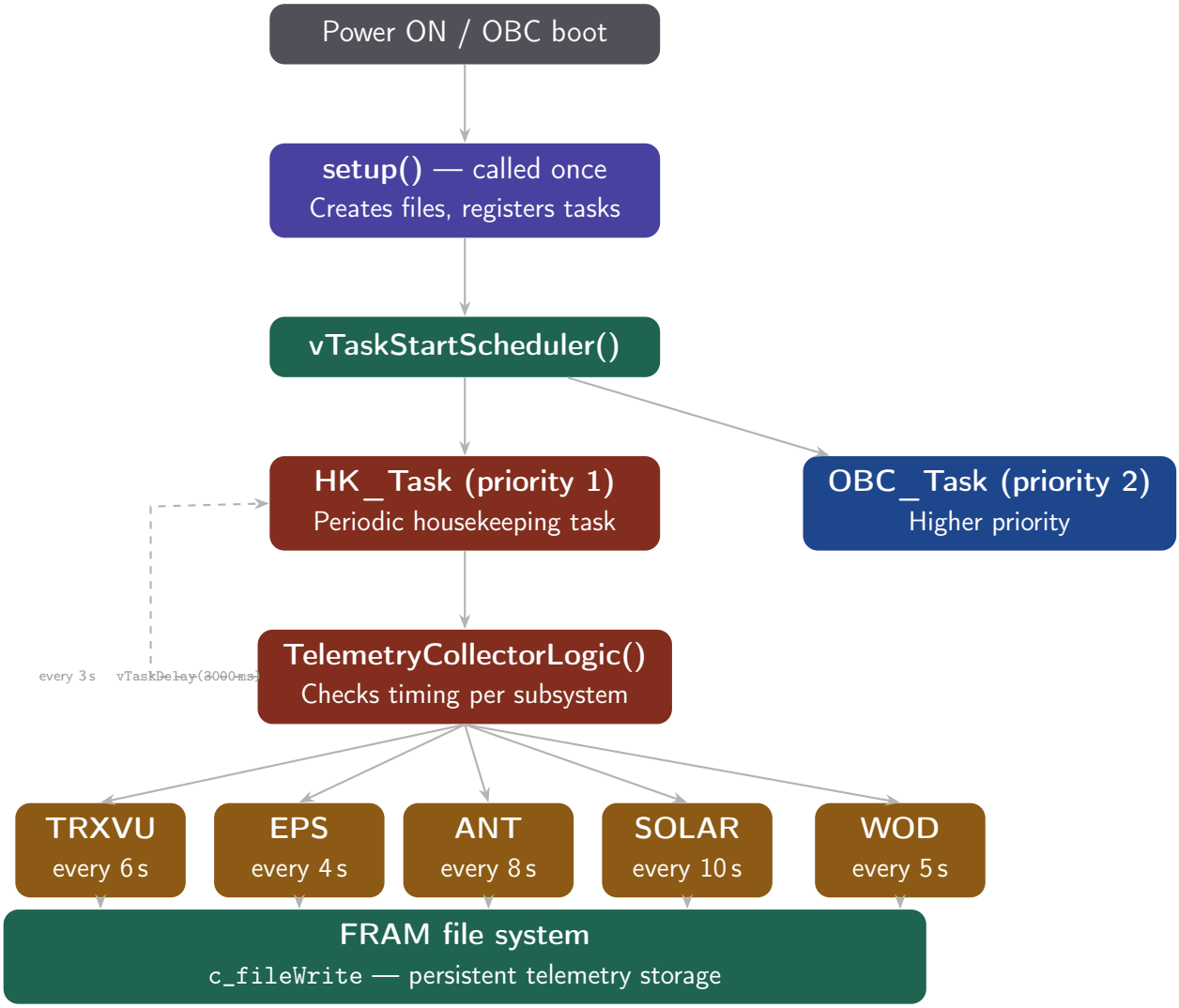
Two FreeRTOS tasks were created to simulate the interaction between the Housekeeping module and the On-Board Computer:

Table 3: FreeRTOS tasks created in the simulation

Task name	Priority	Stack (words)	Role
`OBC_Task`	2 (high)	512	Collects telemetry, sends WOD to queue
`HK_Task`	1 (low)	512	Receives WOD from queue, executes ‘TelemetryCollectorLogic()‘

`OBC_Task` has a higher priority than `HK_Task`, ensuring that telemetry collection is never blocked by the display task. Both tasks are created in `setup()` and the FreeRTOS scheduler is started immediately via `vTaskStartScheduler()`. The `loop()` function is left empty, as the scheduler takes full control of execution after startup.

Call Sequence and Periodicity



8.3 Task Periodicity and Orchestration

`HK_Task` receives a WOD packet from the queue and then calls `TelemetryCollectorLogic()` every 3 seconds via `vTaskDelay(pdMS_TO_TICKS(3000))`. Inside `TelemetryCollectorLogic()`, each subsystem is saved independently based on its own configurable period stored in `tlm_save_periods[]`. The function `CheckExecutionTime()` compares the elapsed time since the last save against the configured period, and triggers the corresponding save function only when the period has elapsed.

Table 4: Telemetry save periods configured in the simulation

Subsystem	Period (s)	Save function called
EPS	4	`TelemetrySaveEPS()`
TRXVU	6	`TelemetrySaveTRXVU()`
Antenna	8	`TelemetrySaveANT()`
Solar panels	10	`TelemetrySaveSolarPanels()`
WOD	5	`TelemetrySaveWOD()`

This design ensures that each subsystem is sampled at its own rate, independent of the others, which is consistent with the real iOBC implementation where `tlm_save_periods[]` is loaded from FRAM at startup via `InitTelemetryCollector()`.

8.4 Inter-task Communication via Queue

A FreeRTOS queue was used to transfer the `WOD_Telemetry_t` structure from `OBC_Task` to `HK_Task`.
The queue was created with a capacity of 5 messages, each of size `sizeof(WOD_Telemetry_t)`.

- `OBC_Task` fills the `WOD_Telemetry_t` struct directly with simulated subsystem values, then sends it to the queue via `xQueueSend()`.
- `HK_Task` blocks on `xQueueReceive()` with `portMAX_DELAY`, waking up only when new data is available.

This mechanism ensures that the two tasks operate independently without shared memory conflicts, which is a fundamental requirement of real-time multitasking on the iOBC.

8.5 Simulation Results

The sketch was uploaded to the Arduino Mega 2560 and the output was observed on the Arduino IDE Serial Monitor at 9600 baud. The following behaviour was confirmed:

- `HK_Task` executes every 3 seconds as configured by `vTaskDelay()`.
- Each subsystem save function is triggered at its own independent period, confirming the correct behaviour of `TelemetryCollectorLogic()`.
- The `WOD_Telemetry_t` structure is correctly populated with all 11 fields by `OBC_Task` before being placed in the queue.

- The queue successfully transfers the WOD packet from `OBC_Task` to `HK_Task` without data loss.
- Task priority is respected: `OBC_Task` always executes before `HK_Task` when both are ready.

Table 5: Example WOD telemetry output observed on Serial Monitor

Field	Example value	Unit
vbat	3842	mV
volt_3V3	3301	mV
volt_5V	4987	mV
current_3V3	143	mA
current_5V	211	mA
charging_power	512	mW
consumed_power	287	mW
free_memory	347821	bytes
corrupt_bytes	2	bytes
number_of_resets	3	–
sat_time	12	sec (since boot)

These results validate the orchestration logic of the HK module and confirm that the `TelemetryCollectorLogic()` function, the FreeRTOS task configuration, and the queue-based communication mechanism all behave correctly in a real-time multitasking environment.

Excerpt: Task Initialization

```
void setup()
{
    Serial.begin(9600);
    randomSeed(analogRead(0)); // seed random from floating pin

    xHKQueue = xQueueCreate(5, sizeof(WOD_Telemetry_t));

    xTaskCreate(OBC_Task, "OBC_Task", 512, NULL, 2, NULL); // higher priority
    xTaskCreate(HK_Task,  "HK_Task",  512, NULL, 1, NULL); // lower priority

    vTaskStartScheduler();
}
```

note : The complete source code associated with this project (Python and C) is publicly available and can be accessed through the following GitHub repository: `https://github.com/ryoucef680-collab/nano-satellite-hk-module`

8.6 WOD Telemetry Logging to CSV File

8.6.1 Principle

The Arduino Mega transmits the WOD telemetry fields over the USB Serial port at 9600 baud. The `HK_Task`, after receiving the `WOD_Telemetry_t` struct from the queue, outputs a structured comma-separated line prefixed with the keyword `WOD`:

```
WOD,<sat_time>,<vbat>,<volt_3V3>,<volt_5V>,<current_3V3>,
    <current_5V>,<charging_power>,<consumed_power>,
    <free_memory>,<number_of_resets>
```

The Python script reads this line, detects the `WOD` prefix, and writes the fields as one row in the CSV file. This approach is reliable because the structured format eliminates any text parsing ambiguity.

Tools Used

Table 6: Tools used for CSV data logging

Tool	Role
Python 3	Script execution on Windows laptop
`pyserial`	Read Serial data from Arduino via USB
`csv`	Write structured rows to `.csv` file
`datetime`	Generate timestamped filename

8.6.2 Script Description

A Python script was developed using the `pyserial` library to read the Serial output of the Arduino Mega on the Windows laptop. The script opens the Serial port at 9600 baud, reads each incoming line, and filters only the lines prefixed with `WOD`. Each detected line is split by comma separator and written as one row in the CSV file. A timestamped filename is generated automatically using `datetime` to avoid overwriting previous files. The `flush()` function is called after each row to ensure data is saved to disk immediately, even if the script is interrupted.

CSV Output Example

The resulting `.csv` file has the following structure:

Table 7: Example content of the WOD telemetry CSV file

sat_time	vbat	volt_3V3	volt_5V	curr_3V3	curr_5V	chg_pwr	cons_pwr	free_mem	resets
12	3842	3301	4987	143	211	512	287	347821	3
15	3756	3298	4991	151	198	487	301	347650	3
18	3901	3305	4978	138	224	521	279	347432	3
21	3812	3310	4982	162	187	498	312	347210	3

Each row corresponds to one WOD telemetry snapshot collected by the `HK_Task` every 3 seconds. This file can be opened directly in Microsoft Excel or any spreadsheet application for analysis and visualization.

9 Conclusion

This study of the GlobusSat project provided a concrete reference for understanding the architecture of a CubeSat housekeeping module built on the ISIS iOBC platform with FreeRTOS. By analysing the available source files, the key functions required for telemetry collection, file creation, and data persistence were identified and documented. The GlobusSat codebase demonstrates a modular and extensible approach to satellite housekeeping, where each subsystem — EPS, TRXVU, antenna, solar panels, and WOD — is handled by a dedicated save function writing records to a structured FRAM file system. This architecture serves as a solid foundation for the development of the HK module for the nanosatellite.

A more advanced version of this project, GlobusSat-2, has also been identified. It extends the original architecture with additional subsystems, improved error handling, and a more complete WOD telemetry implementation. GlobusSat-2 represents a valuable reference for future development phases of the HK module for nanosatellite.

References

[1] GlobusSat Team, *GlobusSat (v1) — Original project*, Available: https://github.com/Rogozin-Tevel/GlobusSatProject/blob/master/GlobusSatProject/src/SubSystemModules/Housekeeping/TelemetryCollector.c

[2] GlobusSat Team, *GlobusSat-2 (v2) — Extended version*, Available: https://github.com/Rogozin-Tevel/GlobusSatProject-2.0/blob/d9581d09df3a7ad829da76a976a0bb0d0dbcc9b9/GlobusSatProject/Hoopoe3/src/sub-systemCode/Main/HouseKeeping.c

[3] ISISpace, *iOBC On-Board Computer*, Available: https://www.isispace.nl/product/on-board-computer

[4] ISISpace, *iEPS Electrical Power System*, Available: https://www.isispace.nl/product/electrical-power-system

[5] ISISpace, *TRXVU Transceiver*, Available: https://www.isispace.nl/product/transceiver

[6] FreeRTOS, *Official Documentation*, Available: https://www.freertos.org/Documentation

[7] HCC Embedded, *FAT File System Library*, Available: https://www.hcc-embedded.com

[8] European Cooperation for Space Standardization (ECSS), *ECSS-E-ST-70-41C: Telemetry and Telecommand Packet Utilization*, 15 April 2016. Available: https://ecss.nl/standards/active-standards/

[9] Alén Space / Satsearch Blog, *A Guide to Command and Data Handling (CDH) Systems*, 2022. Available: https://blog.satsearch.co

[10] eoPortal Directory, *nSight-1 CubeSat — QB50 Mission*, 2017. Available: https://www.eoportal.org/satellite-missions/nsight-1